

Opti Crop: Smart Agricultural Production Optimization Engine

Project Description

Opti Crop: Smart Agricultural Production Optimization Engine is an intelligent web based application designed to assist farmers and agricultural professionals in selecting the most suitable crop based on soil and environmental conditions. The system leverages Machine Learning to analyze key agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall, and recommends the crop that is most likely to achieve higher productivity.

The application is built using **Python, Flask, HTML, CSS, and JavaScript**, with a trained Machine Learning model that provides accurate crop recommendations from user-provided input data. The user-friendly interface enables farmers to enter field information quickly and receive instant recommendations, helping them make informed agricultural decisions.

Opti Crop aims to improve crop yield, optimize resource utilization, reduce production risks, and promote sustainable farming practices. By integrating modern technologies with agriculture, the system minimizes guesswork in crop selection and supports data-driven farming. This contributes to increased agricultural productivity, better soil management, efficient use of fertilizers and water, and improved profitability for farmers.

Scenarios

Scenario 1

A farmer enters soil nutrient values (Nitrogen, Phosphorous, and Potassium) along with environmental conditions such as temperature, humidity, pH level, and rainfall. The system analyzes the inputs and instantly recommends the most suitable crop for achieving higher yield and efficient farming.

Scenario 2

An agricultural researcher evaluates different combinations of soil and climate conditions to study crop suitability. OptiCrop predicts the most appropriate crop and helps identify patterns that support agricultural research and sustainable farming strategies.

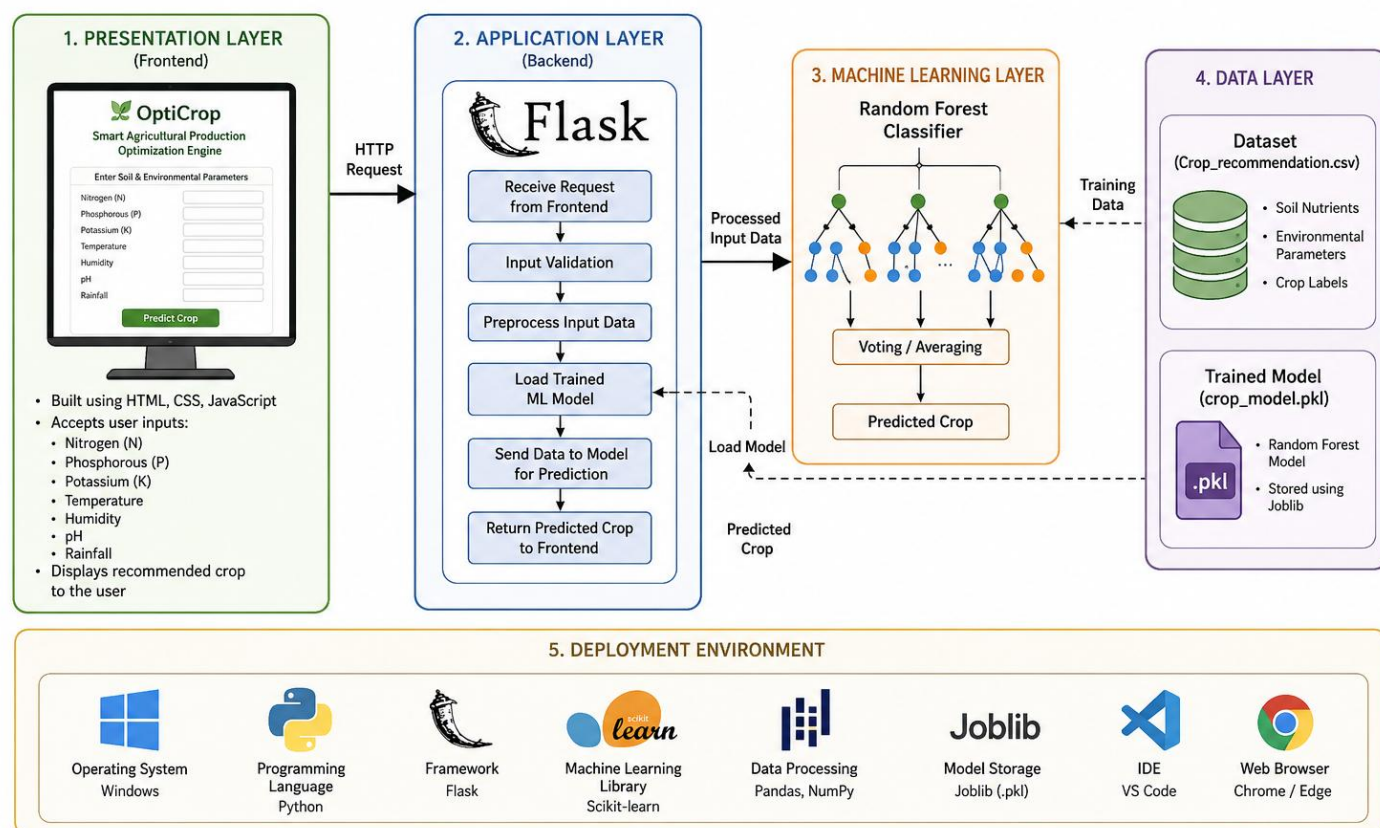
Scenario 3

An agribusiness consultant or policymaker uses OptiCrop to assess crop suitability across different environmental conditions. The platform provides intelligent crop recommendations that assist in resource planning, improving agricultural productivity, and supporting evidence-based farming decisions.

Technical architecture:

Technical Architecture

OptiCrop: Smart Agricultural Production Optimization Engine



Description:

Opti Crop utilizes a modular three-tier architecture to provide intelligent crop recommendation using machine learning. The frontend offers a responsive and user-friendly interface developed with HTML, CSS, and Bootstrap, allowing users to enter soil nutrient and environmental parameters. The Flask-based backend performs input validation, data preprocessing, and prediction by loading a trained Random Forest machine learning model stored in Pickle (.pkl) format. The application processes parameters such as Nitrogen (N), Phosphorous (P), Potassium (K), temperature, humidity, pH, and rainfall to recommend the most suitable crop. The system is developed locally using Flask and can be deployed on cloud platforms such as Render or executed through the Flask development server for real-time crop prediction.

Pre-requisites:

- 1. Python Programming Knowledge:** Python Documentation
- 2. Flask Framework Knowledge:** Flask Documentation
- 3. Machine Learning Fundamentals:** Scikit-learn Documentation
- 4. NumPy and Pandas for Data Processing:** NumPy & Pandas Documentation
- 5. Data Visualization:** Matplotlib & Seaborn Documentation

6. **HTML, CSS, and Bootstrap Skills:** W3Schools HTML/CSS/Bootstrap Tutorials

7. **Jupyter Notebook / Anaconda Environment:** Anaconda Documentation

8. **Version Control with Git and GitHub:** Git Documentation

9. **Flask Application Deployment (Local/Render):** Flask Deployment Guide

Project Workflow

Milestone 1: Project Setup and System Design

- **Activity 1.1:** Set up the project structure in Visual Studio Code and create a Python virtual environment with the required libraries including Flask, Pandas, NumPy, Scikit-learn, Joblib, and Matplotlib.
- **Activity 1.2:** Study the Crop Recommendation dataset to identify input features such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.
- **Activity 1.3:** Design the system architecture showing the interaction between the web interface, Flask backend, machine learning model, and prediction module.
- **Activity 1.4:** Define the workflow for collecting user inputs, processing the data, predicting the suitable crop, and displaying the recommendation.

Milestone 2: Data Preparation and Model Development

- **Activity 2.1:** Clean and preprocess the agricultural dataset by checking missing values and preparing the data for model training.
- **Activity 2.2:** Split the dataset into training and testing sets and train a **Random Forest Classifier** to predict the most suitable crop.
- **Activity 2.3:** Evaluate the trained model for prediction accuracy and save the trained model as **crop_model.pkl** using Joblib for future use.

Milestone 3: Flask Backend Development

- **Activity 3.1:** Develop the Flask application and create routes for the Home, About, Find Your Crop, Services, and Contact pages.
- **Activity 3.2:** Implement the prediction module to receive user inputs, validate the values, and send them to the trained machine learning model.
- **Activity 3.3:** Load the saved Random Forest model and generate crop recommendations based on soil nutrients and environmental conditions.

Milestone 4: User Interface Development

- **Activity 4.1:** Design an attractive and responsive web interface using HTML, CSS, and JavaScript.
- **Activity 4.2:** Develop the "Find Your Crop" page with input fields for Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall.
- **Activity 4.3:** Display the predicted crop dynamically on the result page with a user-friendly layout.

Milestone 5: System Testing and Deployment

- **Activity 5.1:** Test the application by entering different agricultural input values and verify the prediction results.
- **Activity 5.2:** Fix interface issues, validate input handling, and ensure smooth communication between the frontend and backend.
- **Activity 5.3:** Run the application on the local Flask development server and prepare the project for deployment.

Milestone 6: conclusion

- Evaluate the performance of the crop recommendation system using the trained Random Forest model.
- Verify that the application provides accurate crop recommendations based on soil and climate parameters.
- Document the project architecture, implementation process, testing results, and future enhancements.
- Deliver a web-based intelligent crop recommendation system that helps farmers select suitable crops for improved agricultural productivity.

Milestone 1:

Milestone 1: Model Selection and Architecture

In the first milestone, the Opti Crop: Smart Agricultural Production Optimization Engine was planned by analysing the crop recommendation dataset and identifying the key input parameters, including Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall. The Random Forest Classifier algorithm was selected because it provides high prediction accuracy, effectively handles multiple input features, and is robust against overfitting. Based on the selected algorithm, the system architecture was designed by integrating the HTML, CSS, and JavaScript frontend with the Flask backend, data preprocessing module, and the trained Random Forest model to generate accurate crop recommendations efficiently.

Activity 1.1: Environment Setup and Library Installation

Before developing the Opti Crop application, configure the Python development environment and install all required libraries.

```
</> Bash

# Create a Virtual Environment
python -m venv venv

# Activate the Virtual Environment
venv\Scripts\activate

# Install Required Packages
pip install flask numpy pandas scikit-learn joblib gunicorn

# Check Installed Packages
pip list

# Generate Requirements File
pip freeze > requirements.txt

# Train the Machine Learning Model
python train_model.py

# Run the Flask Application
python app.py

# Deactivate the Virtual Environment
deactivate
```

Activity 1.2: Dataset Analysis and Model Selection

The OptiCrop system uses an agricultural dataset containing soil nutrients and environmental parameters.

Dataset Features

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

Output

- Recommended Crop

Steps Performed

- Load the dataset using Pandas
- Perform data cleaning and preprocessing.
- Handle missing values (if any).
- Perform Exploratory Data Analysis (EDA).
- Split the dataset into training and testing sets.
- Train multiple machine learning algorithms.

Algorithms evaluated include:

- K-Nearest Neighbors (KNN)
- Logistic Regression
- Decision Tree
- Random Forest
- K-Means Clustering

After comparing prediction accuracy and evaluation metrics, Random Forest is selected as the final model because it provides the highest prediction accuracy and stable performance for crop recommendation.

Activity 1.3: Define the Application Architecture

Create an architectural diagram representing the interaction between different components of the OptiCrop system.

The architecture consists of:

- User Interface developed using HTML, CSS, and Bootstrap
- Flask Backend for request handling and prediction processing.
- Data Preprocessing module for validating and scaling input values.
- Trained Random Forest Model for crop prediction.
- Prediction Result module for displaying the recommended crop to the user.

Workflow

User → User Interface (HTML, CSS, Bootstrap) → Flask Backend (Request Handling & Prediction Processing) → Data Preprocessing (Input Validation & Feature Scaling) → Trained Random Forest Model → Prediction Result Module (Display Recommended Crop) → Recommended Crop to User

Activity 1.4: Development Environment Configuration

Create the project directory structure

```
OptiCrop
├── app.py
├── train_model.py
├── requirements.txt
├── data
│   └── Crop_recommendation.csv
├── model
│   ├── crop_model.pkl
│   └── scaler.pkl
├── templates
│   ├── index.html
│   ├── about.html
│   ├── services.html
│   ├── findyourcrop.html
│   └── result.html
├── static
│   ├── css
│   │   └── style.css
│   ├── js
│   │   └── script.js
│   └── images
│       ├── home.jpg
│       ├── about.jpg
│       ├── findyourcrop.jpg
│       └── logo.png
└── README.md
```

Required Python Libraries

- Flask
- Scikit-learn
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Pickle

Development Tools

- Python 3.10+
- Jupyter Notebook
- Visual Studio Code
- Git & GitHub

Milestone 1 Outcome

At the end of this milestone:

- The development environment is successfully configured.
- The agricultural dataset is analysed and pre-processed.
- Multiple machine learning algorithms are trained and evaluated
- The Random Forest model is selected as the final prediction model.
- The application architecture is finalized.
- The trained model is saved in .pkl format for integration with the Flask web application.

Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the Opti Crop application are developed. The focus is on preprocessing agricultural data, training the machine learning model, integrating the trained model with the Flask backend, and generating intelligent crop recommendations based on user-provided soil and environmental parameters.

Activity 2.1: Develop the Crop Recommendation Model

Implement the machine learning model responsible for predicting the most suitable crop.

Load the Agricultural Dataset

- Read the agricultural dataset using Pandas.

- Verify the dataset for missing values and inconsistencies.
- Explore the dataset using summary statistics and visualization techniques.

Data Preprocessing

Perform the following preprocessing steps before training:

- Remove duplicate records.
- Handle missing values (if any).
- Separate input features and target labels.
- Apply feature scaling using **StandardScaler**.
- Split the dataset into training and testing sets.

Train Multiple Machine Learning Models

Train and compare different machine learning algorithms:

- K-Nearest Neighbours (KNN)
- Logistic Regression
- Decision Tree
- Random Forest
- K-Means Clustering

Model Evaluation

Evaluate each model using:

- Accuracy Score
- Confusion Matrix
- Classification Report
- Cross Validation

After evaluation, select the Random Forest model because it achieved the highest prediction accuracy :

Save the model:

Store the trained model and scaler using Pickle for future predictions.

Python

```
import pickle
```

```
with open("model/crop_model.pkl", "wb") as file:  
    pickle.dump(model, file)
```

```
with open("model/crop_model.pkl", "rb") as file:  
    model = pickle.load(file)
```

Activity 2.2: Implement the Flask Backend

Develop the Flask backend to process user requests and interact with the trained machine learning model.

Define Flask Routes

Create the following routes in app.py:

- / → Displays the Opti Crop homepage.
- /predict → Receives user inputs and returns the predicted crop.

Python

```
@app.route("/")
```

```
def home():
```

```
    return render_template("index.html")
```

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    # Prediction logic goes here
```

```
    return render_template("result.html", prediction=result)
```

Process User Input

Accept the following parameters from the user:

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

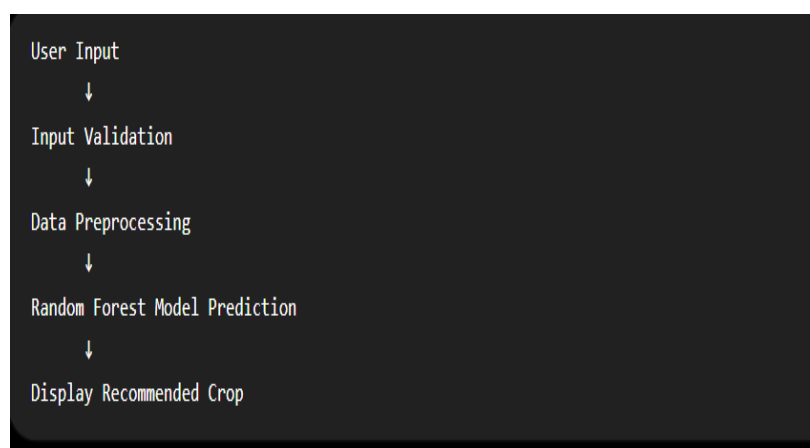
Validate the inputs to ensure:

- No empty values
- Numeric values only
- Values fall within acceptable agricultural ranges

Data Preprocessing

Before prediction:

- Convert input values into numerical format.
- Scale the features using the saved StandardScaler.
- Arrange the values in the correct feature order.



Generate Crop Recommendation

Load the trained Random Forest model

```
</> Python

model = joblib.load("model/crop_model.pkl")
```

Pass the processed input to the model

```
</> Python

prediction = model.predict(input_data)
```

Return the predicted crop to the frontend.

Error Handling

Implement exception handling for:

- Invalid user inputs
- Missing values
- Prediction failures
- Unexpected server errors

```
Python

try:
    prediction = model.predict(input_data)
except Exception:
    return "Prediction Failed"
```

Milestone 2 Outcome

At the end of this milestone:

- Agricultural data is successfully pre-processed.
- Multiple machine learning models are trained and evaluated.
- The Random Forest model is integrated with the Flask backend.
- User inputs are validated before prediction.
- The application generates accurate crop recommendations based on soil nutrients and environmental conditions.
- Error handling ensures stable and reliable application performance.

Milestone 3: App.py Development

Milestone 3 focuses on developing the core logic of the app.py file, which acts as the backbone of the OptiCrop application. In this milestone, Flask routes are created to manage user requests, process agricultural input values, perform data preprocessing, load the trained machine learning model, and generate crop recommendations. Proper input validation and exception handling are also implemented to ensure reliable system performance.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Establish routes for the Opti Crop application, each serving a specific purpose.

Home Route (/)

Displays the main user interface where users can enter soil nutrients and environmental parameters.

</> Python

```
@app.route("/")
def home():
    return render_template("index.html")
```

Prediction Route (/predict)

Accepts user input, validates the values, processes the data, loads the trained Random Forest model, and returns the recommended crop

</> Python

```
@app.route("/predict", methods=["POST"])
def predict():

    if model is None:
        return "Model not found."

    try:
        values = [
            float(request.form["nitrogen"]),
            float(request.form["phosphorous"]),
            float(request.form["potassium"]),
            float(request.form["temperature"]),
            float(request.form["humidity"]),
            float(request.form["ph"]),
            float(request.form["rainfall"])
        ]

        prediction = model.predict([values])[0]
```

</> Python

```
        return render_template(
            "findyourcrop.html",
            prediction=prediction
        )

    except Exception as e:
        return str(e)

if __name__ == "__main__":
    app.run(debug=True)
```

Set Up Route Handlers

For the /predict route, implement logic to read the following agricultural parameters submitted by the user:

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature

- Humidity
- pH
- Rainfall

The application validates all user inputs before passing them to the prediction model.

Validation includes:

- No empty fields
- Numeric values only
- Valid agricultural ranges
- Error handling for invalid inputs

If validation fails, an appropriate error message is displayed to the user.

Load the Trained Machine Learning Model

Load the trained Random Forest model and Standard Scaler during application startup.

```
</> Python

import pickle

model = pickle.load(open("model/crop_model.pkl", "rb"))
scaler = pickle.load(open("model/scaler.pkl", "rb"))
```

These files are generated during the model training phase and reused for real-time prediction

Data Preprocessing

Before prediction, the application preprocesses user inputs by:

- Converting all values into numeric format.
- Arranging features in the correct order.
- Applying feature scaling using the saved Standard Scaler.

Processing workflow:

1. User Input



2. Input Validation



3. Data Preprocessing



4. Pass the Processed Input to the Random Forest Model



5. Display Recommended Crop

Generate Crop Recommendation

The preprocessed input is passed to the trained Random Forest model.

```
</> Python
```

```
prediction = model.predict(input_scaled)
```

The predicted crop is returned and displayed on the result page.

```
Recommended Crop : Rice
```

```
Prediction Status : Successful
```

```
Thank You for Using OptiCrop!
```

Error Handling

Implement exception handling to improve application reliability.

The application handles:

- Invalid numeric inputs
- Missing field values
- Prediction failures
- Unexpected runtime errors

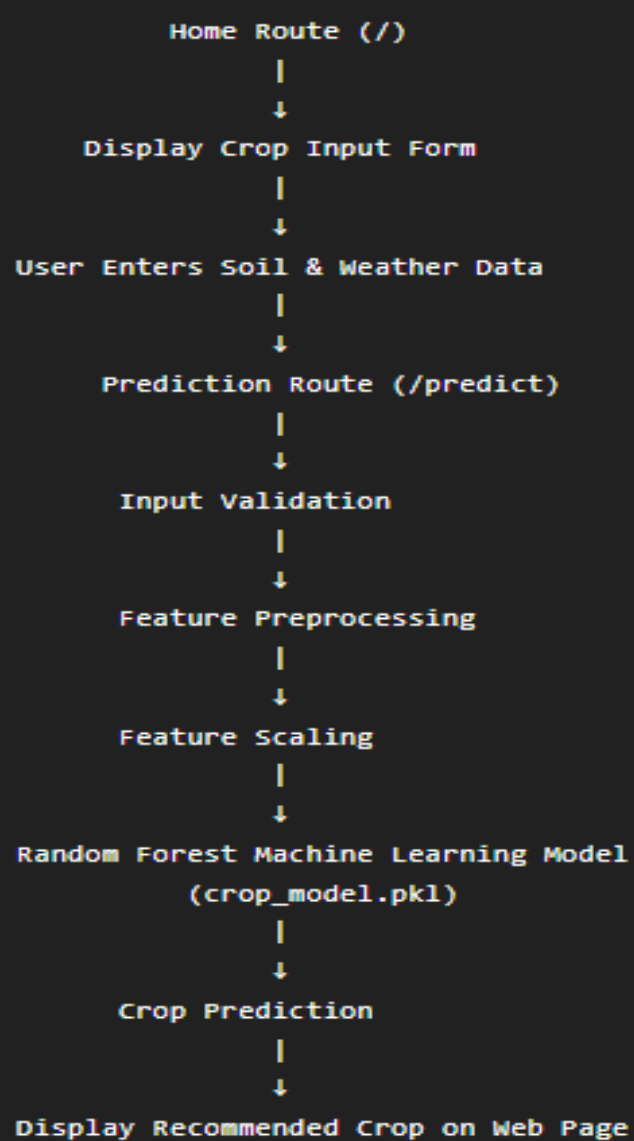
`</>` Python

```
input_scaled = scaler.transform(input_data)

try:
    prediction = model.predict(input_scaled)
    prediction_text = prediction[0]

except Exception:
    prediction_text = "Prediction Failed. Please enter valid values."
```

Application workflow



Milestone 3 Outcome

At the end of this milestone:

- Flask routes are successfully implemented.

- The trained Random Forest model is integrated into the application.
- User inputs are validated before prediction.
- Agricultural data is preprocessed automatically.
- Crop recommendations are generated accurately in real time.
- Proper exception handling ensures stable and reliable application performance.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a responsive, interactive, and user-friendly web interface for the OptiCrop application. The frontend enables users to enter soil nutrient and environmental parameters, submit prediction requests, and view crop recommendations in an organized and visually appealing dashboard. HTML, CSS, Bootstrap, and JavaScript are used to create a modern interface that enhances usability and accessibility.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML file (index.html) as the primary interface for the OptiCrop application
- Create a header containing the OptiCrop project title, logo, and application description.
- Design separate sections for:
 - Project Overview
 - Agricultural Input Form
 - Crop Recommendation Result
 - Information Cards explaining how the prediction system works.
 - Use semantic HTML elements to improve readability, maintainability, and accessibility
 - Integrate Bootstrap components and Font Awesome icons to enhance the appearance of the interface.

Design a Responsive Layout Using CSS

- Create a stylesheet (static/css/style.css) to provide a clean and modern user interface.
- Apply responsive design principles using Bootstrap Grid and Flexbox layouts.
- Organize the dashboard into multiple sections including:
 - o Project Information
 - o Machine Learning Model Details
 - o Agricultural Parameter Input Form
 - o Prediction Result
- Use media queries to ensure compatibility across desktop, tablet, and mobile devices.

Create Separate UI Components

Develop individual interface components for different functionalities.

Project Dashboard

Display:

- Project Title
- Application Description
- Machine Learning Model Used
- Number of Crop Classes

Input Form

Create input fields for:

- Nitrogen (N)
- Phosphorous (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall

Provide placeholder text and validation messages for each field.

Prediction Result Card

Display the predicted crop in a highlighted result section after successful prediction.

Example:

Recommended Crop

Rice

Information Section

Display informational cards describing:

- Smart Data Preprocessing
- Machine Learning Prediction
- Decision Support
- Adaptive Crop Recommendations

These cards help users understand how the OptiCrop system works

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission

- Implement the prediction form using HTML forms.
- Accept user inputs for all agricultural parameters.
- Submit the data securely to the /predict route using the POST method.

Display Prediction Results

- After processing the input values, dynamically display the recommended crop on the webpage.
- Present the result in a clearly visible card or alert box.
- If invalid data is entered, display an appropriate error message without crashing the applica

Improve User Experience

Implement additional frontend features such as:

- Clear placeholder text for all input fields.
- Responsive buttons with hover effects.
- Input validation before form submission.
- Attractive Bootstrap cards and icons.
- Easy-to-read typography and spacing.
- Smooth page navigation between the input and result sections.

Responsive Design

Ensure that the Opti Crop interface functions correctly on:

- Desktop Computers
- Laptops
- Tablets
- Mobile Devices

The interface automatically adjusts component sizes and layouts according to screen resolution.

Milestone 4 Outcome

At the end of this milestone:

- A responsive and user-friendly web interface is successfully developed.
- Users can enter soil and environmental parameters through an intuitive input form.
- The application displays crop recommendations dynamically after prediction.
- Bootstrap and CSS provide a clean and professional dashboard layout.
- The frontend integrates seamlessly with the Flask backend, ensuring a smooth user experience across different devices.

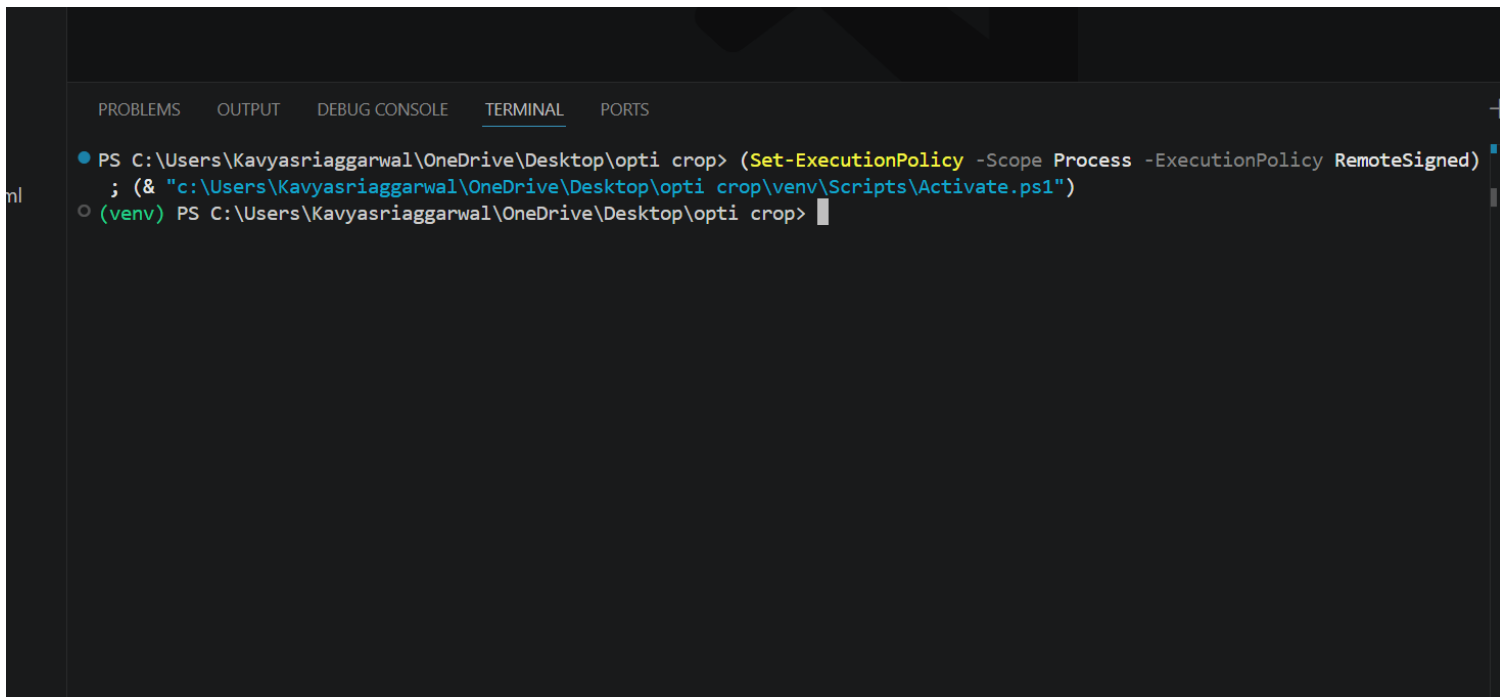
Milestone 5: Deployment

Milestone 5 focuses on deploying the OptiCrop application for testing and demonstration. The application is first executed locally to verify its functionality and then prepared for deployment on a cloud platform such as Render. This milestone ensures that the machine learning model, Flask backend, and frontend interface work together seamlessly in a production-ready environment.

Activity 5.1: Preparing the Application for Local Deployment

- Set Up a Virtual Environment

- Open Command Prompt or Visual Studio Code Terminal.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\Kavyasriaggarwal\OneDrive\Desktop\opti crop> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
; (& "c:\Users\Kavyasriaggarwal\OneDrive\Desktop\opti crop\venv\Scripts\Activate.ps1")
○ (venv) PS C:\Users\Kavyasriaggarwal\OneDrive\Desktop\opti crop> |
```

Create a virtual environment:

</> Bash

```
python -m venv optienv
```

Activate the virtual environment.

Windows

</> Bash

```
optienv\Scripts\activate
```

Install all required dependencies.

</> Bash

```
pip install -r requirements.txt
```

The required libraries include:

- Flask • Scikit-learn • Pandas • NumPy • Matplotlib • Seaborn • Pickle

Configure the Project

Ensure the following files are available in the project directory:

- app.py
- requirements.txt
- crop_recommendation_model.pkl
- scaler.pkl
- dataset.csv
- templates/
- static/

Verify that the trained machine learning model loads correctly before starting the application

Activity 5.2: Local Testing and Verification

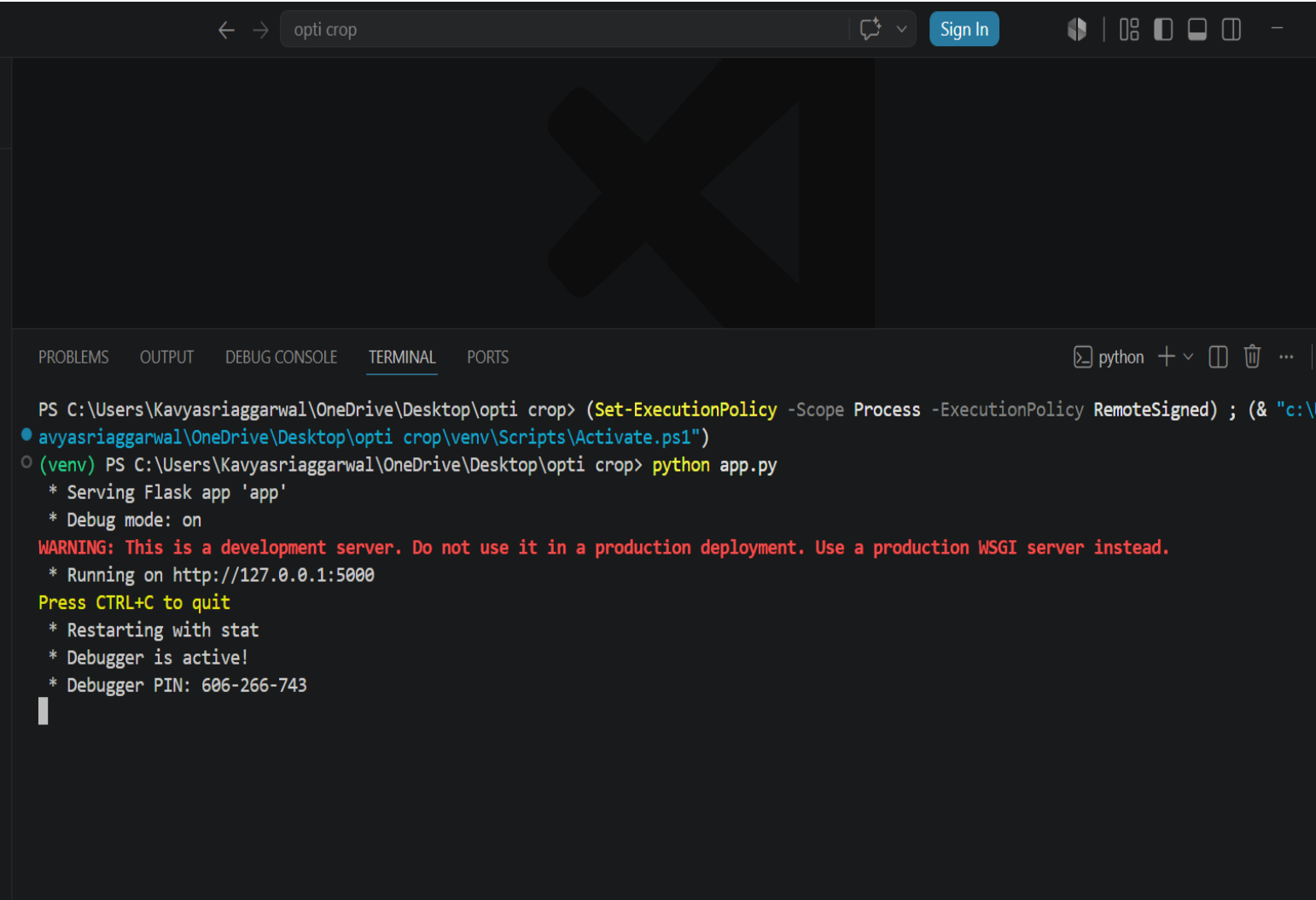
Start the Flask Development Server:

- Run the application locally using:

```
Cd Opticrop

Python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app



Test the Crop Recommendation System	
Enter sample agricultural values:	
Parameter	Sample Value
Nitrogen	90
Phosphorus	42
Potassium	43
Temperature	20.8
Humidity	82
pH	6.5
Rainfall	202

Click **Predict Crop**.

The application processes the input and displays the recommended crop on the webpage.

Repeat testing using different input combinations to verify prediction accuracy and application stability.

Activity 5.3: Deploying the Application

Deploy the OptiCrop application using Render.

Prepare the Project

Ensure the following files are included:

- app.py
- requirements.txt
- runtime.txt (optional)
- models/
- templates/
- static/

Upload the complete project to a GitHub repository

Deploy on Render

1. Sign in to Render.
2. Select New Web Service.
3. Connect your GitHub repository.
4. Select the OptiCrop repository.
5. Configure the deployment.

Build Command


```
</> Bash
```

```
pip install -r requirements.txt
```

```
</> Bash
```

```
python app.py
```

Click **Deploy**.

Render automatically builds and deploys the application.

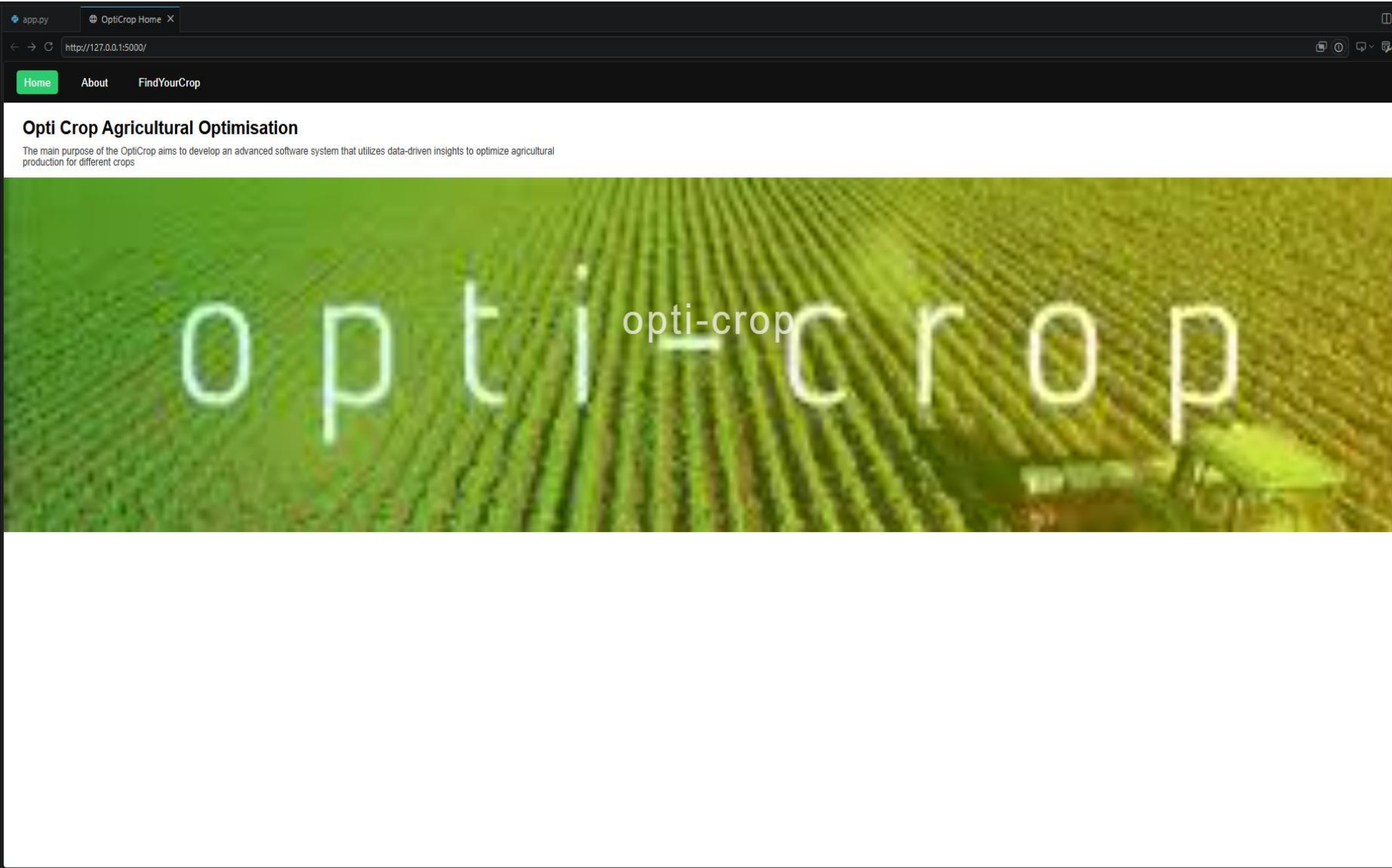
After deployment, Render generates a public URL similar to:

Important Notes +

- Ensure all required Python packages are listed in requirements.txt.
- Keep the trained model (.pkl file) inside the project directory.
- Verify that all template and static files are uploaded.
- Test the deployed application to confirm that predictions work correctly.
- If deployment fails, review the Render build logs and resolve dependency issues.

Exploring the Website's Web Pages:

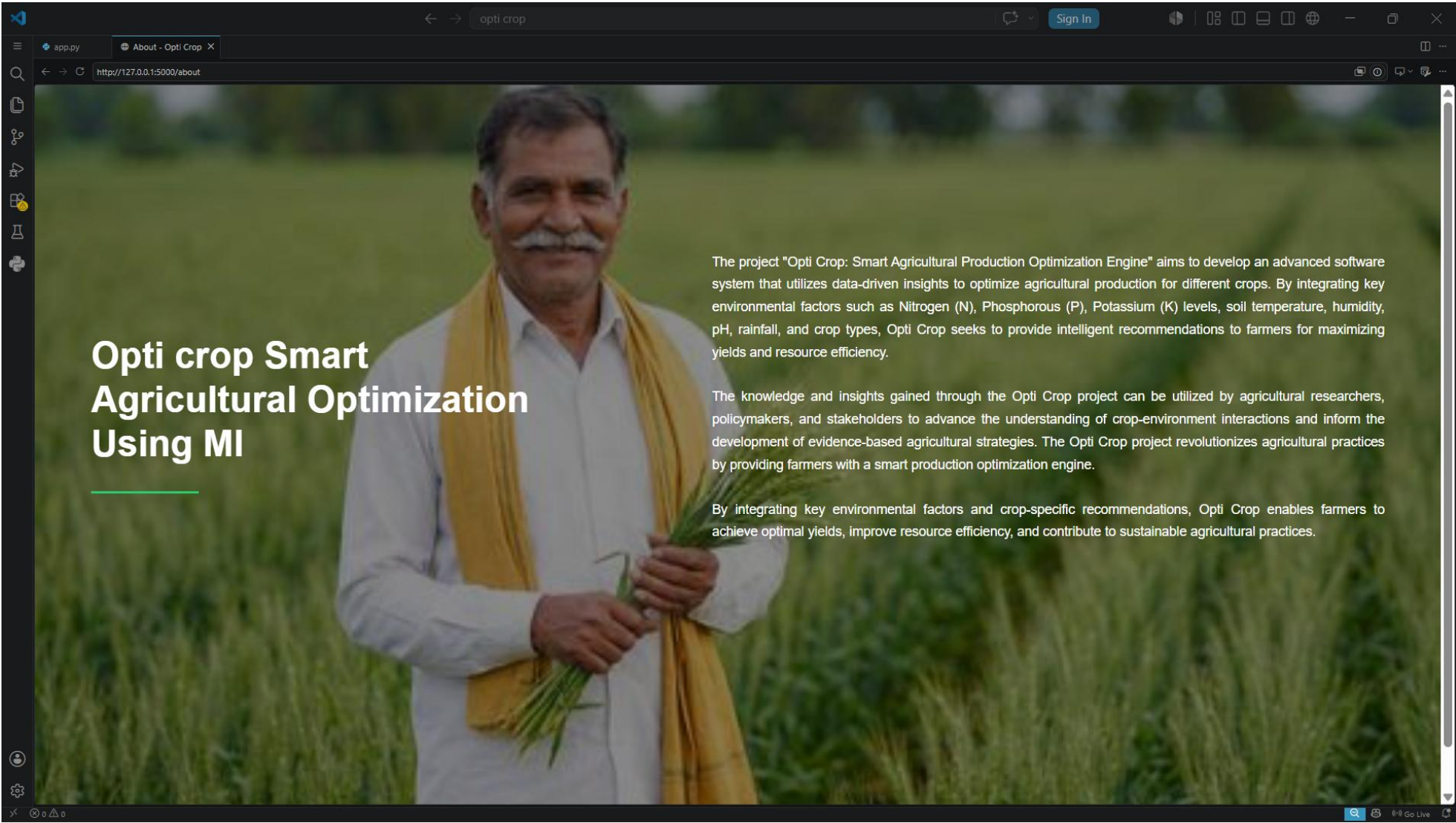
Home / Generator Page:



Description:

Our platform is designed to help farmers make informed crop selection decisions using machine learning and environmental data. By analysing key agricultural parameters such as nitrogen, phosphorus, potassium, temperature, humidity, soil pH, and rainfall, Opti Crop recommends the most suitable crop for cultivation. This enables farmers to improve productivity, optimize resource utilization, reduce risks, and promote sustainable farming practices for better agricultural outcomes.

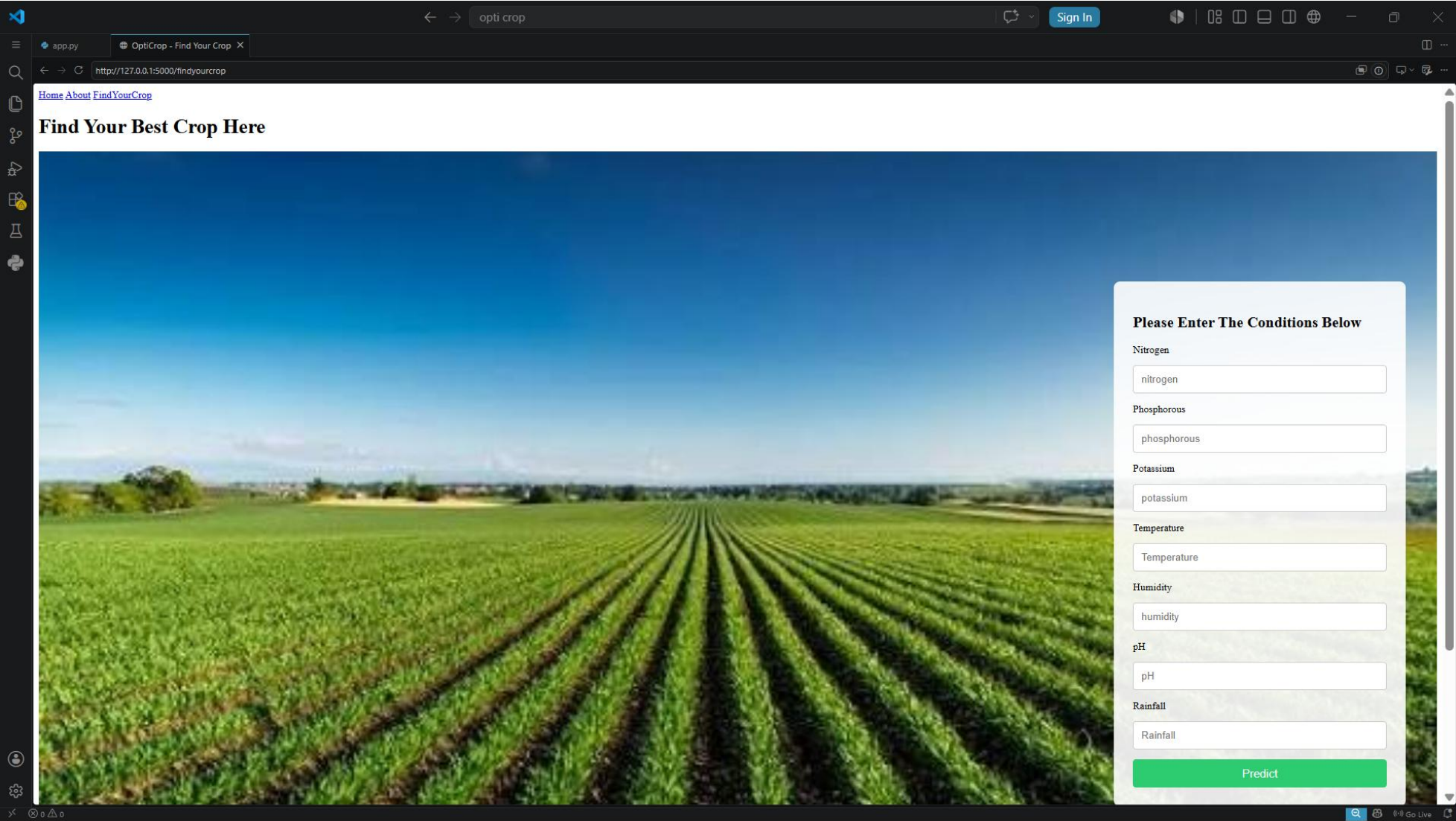
About section:



Description:

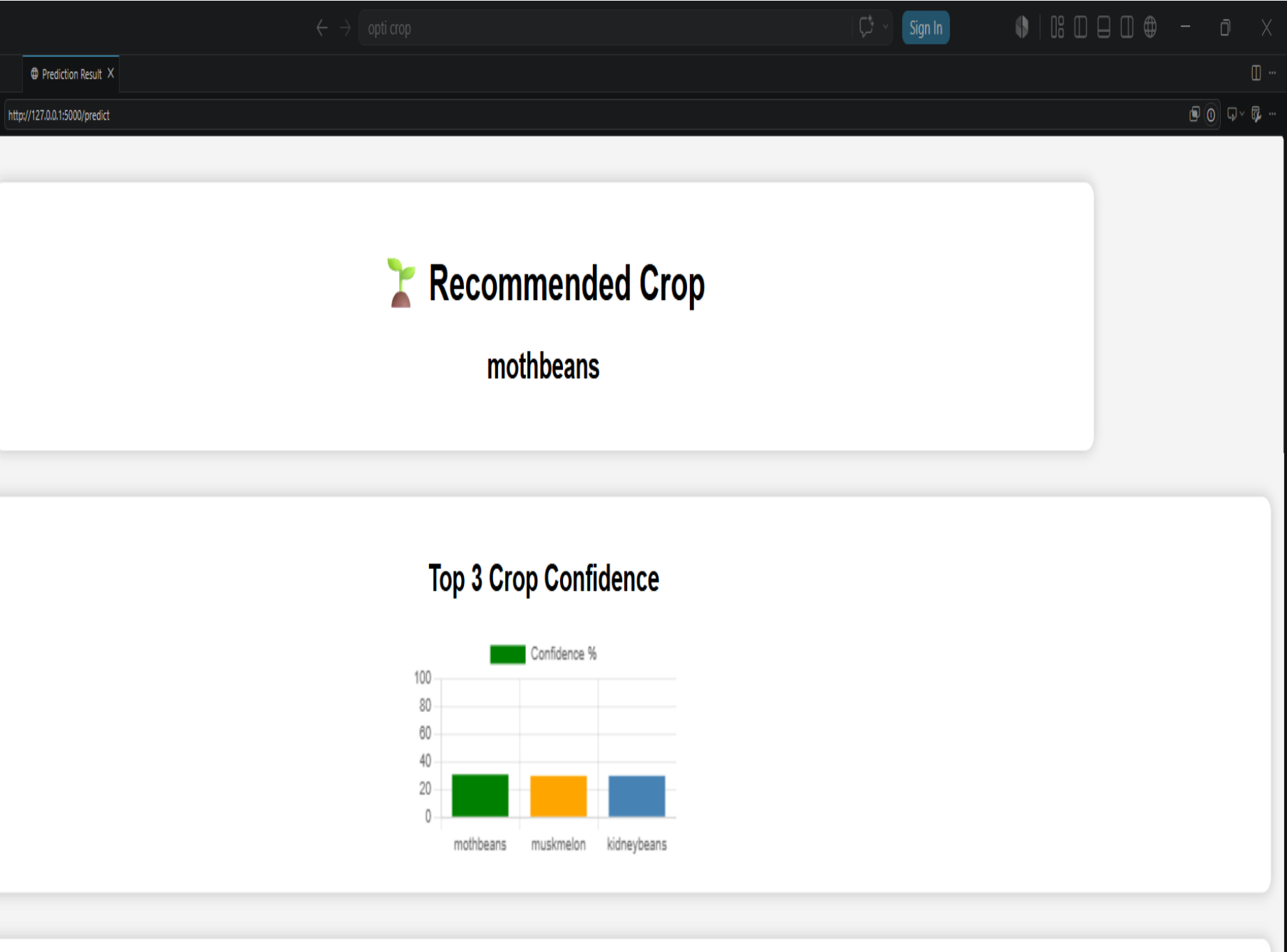
The About section helps users understand the purpose, functionality, and benefits of the OptiCrop system. It explains how machine learning is used to provide accurate crop recommendations, enabling farmers to improve productivity, optimize resources, and adopt sustainable farming practices.

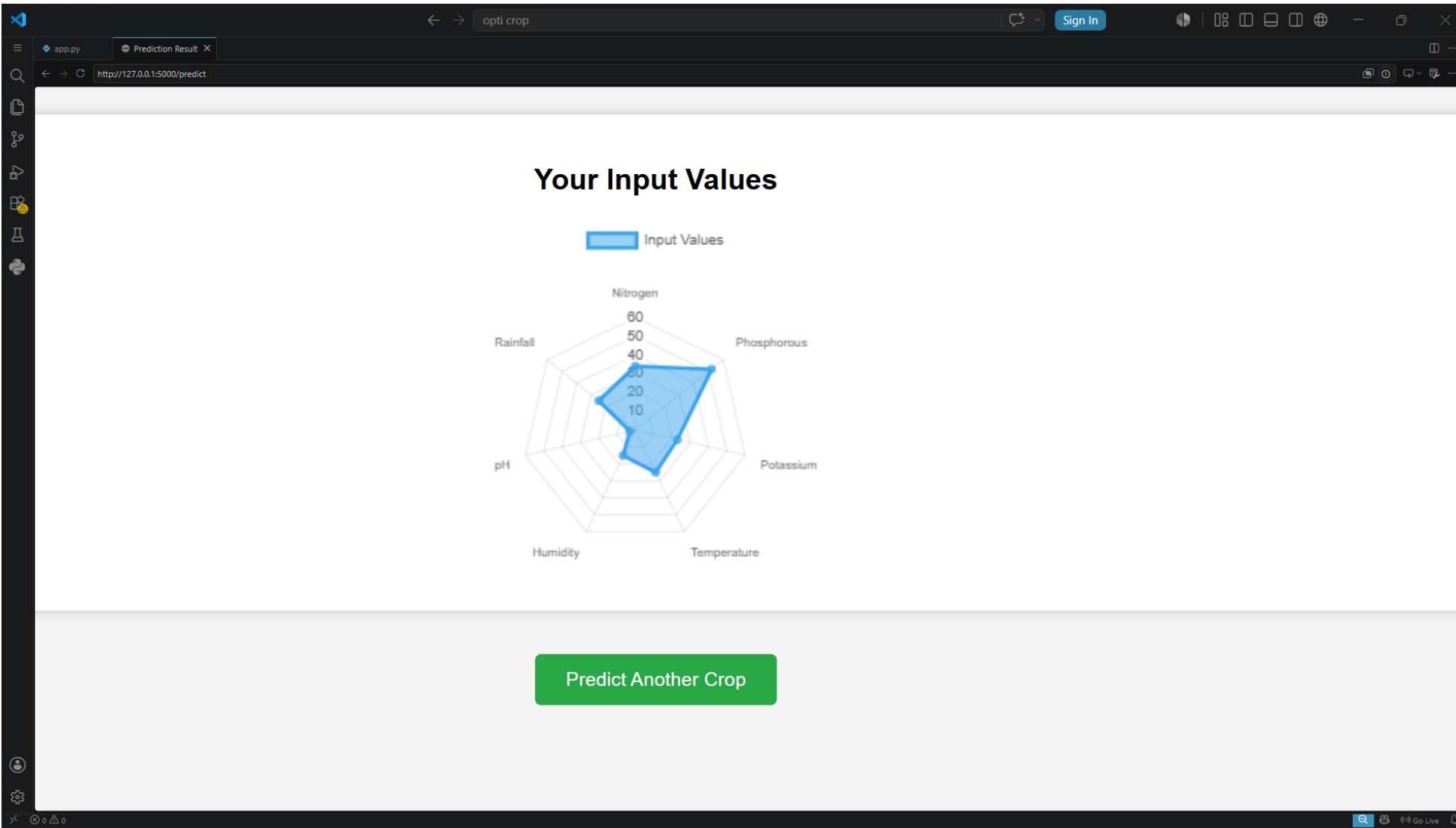
Input Section:



Description:Find Your Crop is the core feature of the Opti Crop system that enables users to identify the most suitable crop based on environmental and soil conditions. Users enter important agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall into the prediction form. The machine learning model analyses these inputs and recommends the best crop for cultivation. This feature helps farmers make informed decisions, improve crop productivity, optimize resource utilization, and support sustainable agricultural practices.

Result section:





Description:

1. Predicted Crop Result

The Result section displays the crop recommended by the OptiCrop machine learning model based on the environmental and soil parameters provided by the user. The prediction helps farmers select the most suitable crop for cultivation, improving productivity and reducing the risk of unsuitable crop selection.

2. Top 3 Crop Confidence

This section presents the top three crop recommendations along with their confidence percentages. The confidence chart shows how strongly the machine learning model predicts each crop, enabling users to compare alternative crops and make informed farming decisions.

3. Your Input Values

The Input Values chart visually represents the agricultural parameters entered by the user, including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall. This graphical representation helps users verify their inputs and understand the factors influencing the crop recommendation.

Conclusion:

OptiCrop: Smart Agricultural Production Optimization Engine is an intelligent crop recommendation system developed to assist farmers in making informed agricultural decisions using machine learning. The application analyses essential parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, soil pH, and rainfall to recommend the most suitable crop for cultivation.

The project was developed using Python with the Flask web framework for the backend, while HTML, CSS, and JavaScript were used to build an interactive and user-friendly frontend. The machine learning model was implemented using the Random Forest Classifier algorithm from Scikit-learn, with Pandas and NumPy used for data preprocessing and analysis, and Joblib for model serialization.

Overall, OptiCrop demonstrates how modern technologies and machine learning can improve agricultural productivity, optimize resource utilization, and support sustainable farming practices. The project provides an efficient, accurate, and easy-to-use solution that empowers farmers to make data-driven decisions, contributing to higher crop yields and the advancement of smart agriculture.